

# Rozdział 1

## Sortowanie

Co prawda pokazaliśmy, że dolne ograniczenie dla czasu działania dowolnego algorytmu sortującego wynosi  $\Omega(n \log n)$  jednak tylko w modelu drzew decyzyjnym, czyli dotyczy tylko wszystkich algorytmów sortujących według porównań. Jeżeli przyjmiemy inną strategię, dopuścimy randomizację i dane wejściowe mają jakieś szczególne cechy, w których możemy sortować to możemy nieco tę złożoność zbić. Można kombinować z drzwami van Emde Boasa (których operacje wykonywane są w czasie  $O(\log \log n)$ ), z jakimś haszowaniem, jednak istnieją prostsze przykłady.

### 1.1 Sortowanie przez zliczanie - Counting sort

Pamiętam, że na WDI na jednej z list mieliśmy zadanie, w którym trzeba było zaimplementować ten algorytm w kodzie RAM. Gdyby to był chociaż assembler, to nawet potraktowałbym to, jako ciekawą, intelektualną zabawę, tylko z niewielką domieszką masochizmu. Pisanie nawet tak prostego algorytmu w kodzie RAM to czysty masochizm.

Sama idea algorytmu jest prosta: przyjmujemy, że dane na wejściu są liczbami naturalnymi od 0 do  $k-1$ , wobec tego możemy zliczać w pomocniczej tablicy  $C$  wystąpienia każdej z tych liczb w tablicy wejściowej  $A$ , a następnie przepisać  $C$  do  $A$ , wpisując element  $i$  tyle razy ile został zliczony:

```
for  $i \leftarrow 1$  to  $n$  do
```

```
     $C[A[i]]++$  ▷ Zwiększamy licznik wystąpienia elementu  $A[i]$ 
```

```
end for
```

```
 $i \leftarrow 1$ 
```

```
for  $j \leftarrow 0$  to  $k-1$  do
```

```
    while  $C[j] > 0$  do ▷  $\forall j = 0 \dots k-1$  przepisujemy  $j$  tyle razy do  $A$ , ile w  $A$  wystąpił
```

```
         $C[j]--$ 
```

```
         $A[i] \leftarrow j$ 
```

```
         $i++$ 
```

```
    end while
```

```
end for
```

I niby wszystko wygląda ładnie, pierwsza pętla wykona się  $n$  razy, druga wykona  $k+n$  operacji (przeiterujemy

się po wszystkich  $k$ , a łączna liczba obrotów pętli while wyniesie tyle ile jest elementów w tablicy  $A$ , czyli  $n$ ), co da złożoności  $\Theta(n+k)$ . Tylko taki algorytm jest mało użyteczny, bo sortuje tylko po wartościach z małego uniwersum, którym jest podzbiór liczb naturalnych. A my byśmy chcieli sortować stabilnie, różne obiekty, po przypisanym ich kluczom ze zbioru  $\{1, \dots, k-1\}$ , np. słowa, po  $i$ -tej literze.

### Zakuj to!

Algorytm sortujący nazywamy *stabilnym*, jeżeli zachowuje kolejność elementów równych sobie, czyli na przykład dostając tablicę  $[1_1, 5_1, 2_1, 2_2, 4_1, 1_2, 1_3]$ , zwróci  $[1_1, 1_2, 1_3, 2_1, 2_2, 4_1, 5_1]$ , gdzie dolne indeksy po prostu oznaczają, który raz z kolei wystąpił element o danej wartości.

Jak się zaraz przekonamy ta na pozór nieprzydatna własność jest w niektórych przypadkach bardzo istotna. Pomysł jest taki: niech teraz  $C$  będzie taką tablicą, że  $C[i]$  oznacza ilość elementów mniejszych równych  $i$  (tj. o kluczach mniejszych równych  $i$ ), czyli zgrubsza mówimy, że jeżeli  $C[i]$  jest równe jakiemuś  $k$ , to  $i$  jest  $k$ -tym elementem w  $A$  i powinien znaleźć się na  $k$ -tej pozycji w wynikowej tablicy (tutaj  $B$  będzie zawierać wynik sortowania).  $C$ , możemy obliczyć najpierw zliczając elementy (jak wyżej), a potem dynamicznie liczyć  $C[i] \leftarrow C[i] + C[i-1]$ , bo ilość elementów mniejszych równych  $i$ , można rozumieć jako sumę ilości elementów mniejszych równych  $i-1$  oraz ilości elementów równych  $i$ .

Dzięki takiemu na pierwszy rzut oka przekombinowanej modyfikacji jesteśmy w stanie zapewnić stabilność tego sortowania. W jaki sposób? Po skonstruowaniu  $C$  przetwarzamy tablicę  $A$  od tyłu, wówczas  $C[A[i]]$  będzie nam wystaczać pozycję elementu  $A[i]$  w docelowej posortowanej tablicy. Ale chcemy, żeby każde kolejne  $A[j] \cdot j < i$  trafiało na pozycję mniejszą od  $A[i]$ , bo chcemy mieć stabilne sortowanie. Co zatem robimy? Dekrementujemy  $C[A[i]]$ , dzięki czemu kolejny (idąc od tyłu) element równy  $A[i]$  trafi na pozycję o 1 mniejszą.

Pseudokod przeklejam z notatek w xournalpp, w Cormenie i w notatkach Kło jest bardzo podobny.

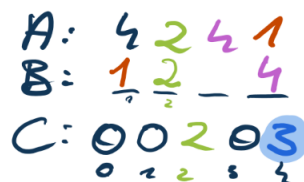
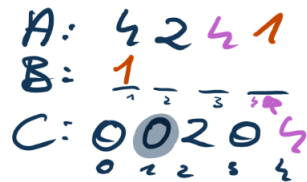
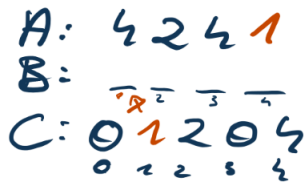
```

for i ← 0 ... k-1 C[i] ← 0
for i ← 1 ... n:
    C[A[i]] ++
for i ← 0 ... k-1
    C[i] ← C[i] + C[i-1]
B ← pusta tablica
for i ← n downto 1:
    B[C[A[i]]] ← A[i]
    C[A[i]] --

```

W miarę przekonujący przykład też przeklejam ze swoich notatek w xournalu. Zróćmy uwagę na to co się dzieje, jak przetwarzamy na przykład  $A[3] = 4$ .  $C[A[3]]$  wynosi 4 co oznacza, że  $A[3] = 4$  jest czwartym elementem i powinno trafić na pozycję czwartą. Ale jednocześnie chcemy, że jeżeli pojawi się kolejny element równy  $A[3]$  - chcemy aby trafił na pozycję  $4 - 1 = 3$ , co dzieje się w przypadku  $A[1]$ , dlatego  $C[A[3]]$  dekrementujemy.

*Przykład*



## 1.2 Sortowanie kubełkowe - Bucket sort

Tutaj wkracza losowość - o  $n$  danych na wejściu zakładamy, że są z rozkładu jednostajnego  $U[0, 1]$ . To oznacza, że są losowymi liczbami z przedziału  $[0, 1]$  oraz że wylosowanie każdej liczby z tego przedziału jest jednakowo prawdopodobne. Idea algorytmu jest prosta, tworzymy  $n$  kubełków (w postaci tablic lub list), takich, że do  $i$ -tego  $\forall i = 0, \dots, n-1$  kubełka trafią liczby z przedziału  $[\frac{i}{n}, \frac{i+1}{n}]$ . Łatwo zauważyć (przynajmniej intuicyjnie), że prawdopodobieństwo, że element trafi do jakiegoś konkretnego kubełka wynosi  $\frac{1}{n}$ , bo mamy  $n$  przedziałów i do każdego element może trafić z jednakowym prawdopodobieństwem. Następnie sortujemy te kubełki, najlepiej jakimś stabilnym sortowaniem, zwykle wybiera się do tego insert-sort (ze względu na sortowanie w miejscu, wspomnianą stabilność i jak się okaże - nie psucie czasu oczekiwanego), następnie łączymy te kubełki w wynikową - posortowaną tablicę.

Na chłopski rozum - pomysł jest kiepski, w najgorszym wypadku wszystkie  $n$  elementów trafi do jednego kubełka i sortowanie zajmie czas kwadratowy. Jednak oczekiwany czas (czyli *statystycznie najczęstszy*) zajmie czas liniowy, pokażemy zatem następujące twierdzenie:

**Twierdzenie 1** *Bucket-sort działa w oczekiwanym czasie  $O(n)$*

*dowód:*

Niech  $X_i$  będzie zmienną losową oznaczającą rozmiar kubełka. Obliczmy prawdopodobieństwo, że rozmiar tego kubełka ma rozmiar  $k$ . Skoro trafiło tam  $k$  elementów, oznacza to, że spośród  $n$  losowo wybranych elementów,  $k$  trafiło do  $i$ -tego kubełka - zatem mamy sytuację, w której przeprowadzamy  $n$  prób i interesuje nas prawdopodobieństwo, że  $k$  zakończyło się sukcesem, zatem mamy do czynienia z rozkładem Bernoullego (konkretnie  $B(n, \frac{1}{n})$ , bo mamy  $n$  prób i prawdopodobieństwo sukcesu - element trafi do  $i$ -tego kubełka wynosi  $\frac{1}{n}$ , jak powiedzieliśmy wynosi  $\frac{1}{n}$ ), stąd:

$$P(X_i = k) = \binom{n}{k} \left(\frac{1}{n}\right)^k \left(\frac{n-1}{n}\right)^{n-k}$$

Nas jednak interesuje  $X_i^2$ , bo taki będzie czas sortowania (kwadratowy względem rozmiaru kubełka), a konkretnie oczekiwana wartość  $E[X^2]$ . Tutaj posłużę się już udowodnionymi z RPiSu prawidłowościami. Wiemy m.in. że  $V[X] = E[X^2] - (E[X])^2 \iff E[X^2] = V[X] + (E[X])^2$ , gdzie  $V[X]$  to wariancja (czyli miara tego, jak bardzo mierzone wartości odstają od wartości oczekiwanej). Ponadto (skoro  $X$ ) podlega rozkładowi Bernoullego, wiemy, że:

$$1. E[X] = n \cdot \frac{1}{n} = 1$$

$$2. V[X] = n \cdot \frac{1}{n} \left(1 - \frac{1}{n}\right) = 1 - \frac{1}{n}$$

Zatem oczekiwana długość sortowania dowolnego kubelka wynosi  $2 - \frac{1}{n}$ . Zatem oczekiwany czas wszystkich sortowań to  $E[X_1^2 + \dots + X_n^2] = E[X_1^2] + \dots + E[X_n^2] = 2n - 1 = O(n)$ . Poza sortowaniem dochodzi scalanie kubków, ale to też zrobimy liniowo.  $\square$

Pseudokod również przeklejam z notatek, nie jest trudny, ale może coś rozjaśni:

1. Rozrzućmy elementy do kubków (el.  $A[i]$ , trafi do kubka  $B[\lfloor n A[i] \rfloor])$   
 2.  $\forall i$  sortujemy kubek  $B[i]$   
 3. Konkatenujemy kubki

### 1.3 Sortowanie leksykograficzne - Radix sort

Chcemy sortować słowa tak jak w słowniku - w kolejności alfabetycznej (czyli w porządku leksykograficznym). Rozważymy dwie wersje tego problemu w zależności od tego, czy wszystkie słowa są tej samej długości, czy też nie.

#### 1.3.1 Słowa tej samej długości